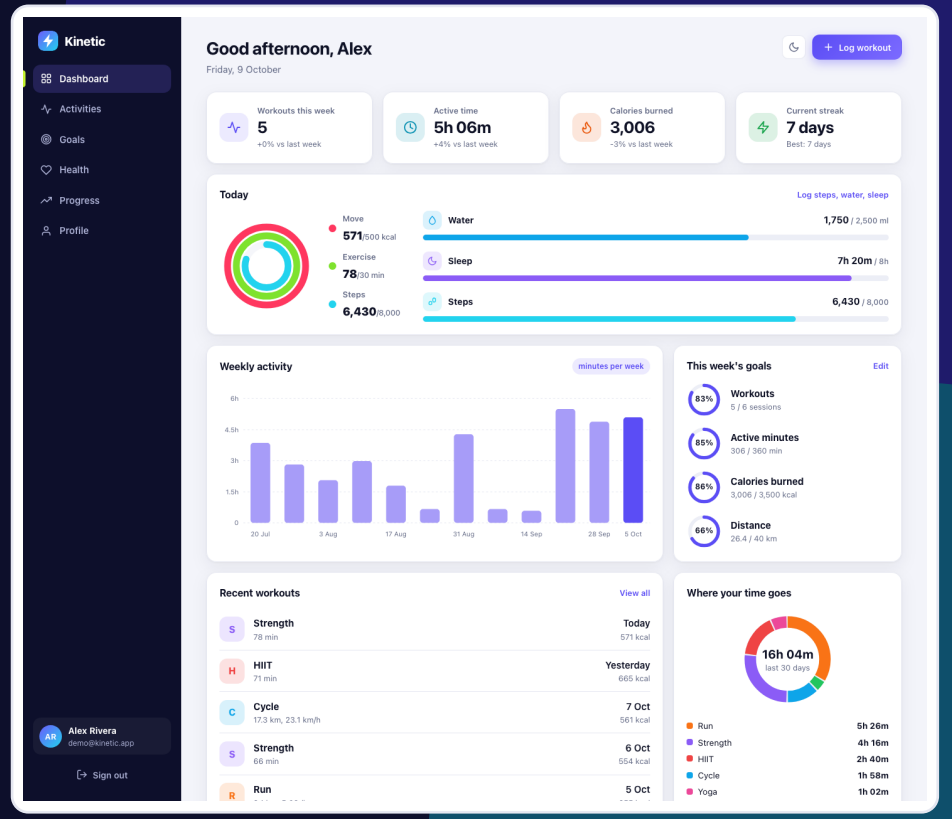
Kinetic

A fitness tracking web application with workouts, weekly goals, streaks and Google sign-in

- **Java Web Project | Servlets, JDBC, core Java**



PROBLEM UNDERSTANDING

The problem and our solution

The problem

- Workout data is scattered across notes, watches and spreadsheets
- People cannot tell if they are improving week to week
- Weekly targets are set but never checked
- Streaks and personal bests are invisible

Our solution: Kinetic

- One log for 7 workout types, with calories estimated from body weight
- Dashboard with weekly totals, trends and time-by-type
- Goal rings for workouts, minutes, calories and distance
- Streaks, activity calendar and personal records
- Fast sign-in with Google or email

Who it is for: anyone who trains regularly and wants clear numbers, not a complicated app. Built as a full Servlet and JDBC web application with no framework.

WHAT IT DOES

Key features

1

Log and edit workouts

Run, walk, cycle, swim, strength, HIIT, yoga. Duration, distance, effort, notes.

2

Smart calories

MET x weight x hours. Each activity class supplies its own MET value.

3

Dashboard

Today rings, week vs last week, 12-week chart, goal rings, time-by-type donut.

4

Weekly goals

Targets for sessions, minutes, calories, distance. Saved in one transaction.

5

Streaks and records

Current and best streak, longest distance, biggest session.

6

Google sign-in

Continue with Google. Token verified on the server before a session starts.

7

Daily health

Steps, water and sleep with daily rings, plus a body-weight trend line.

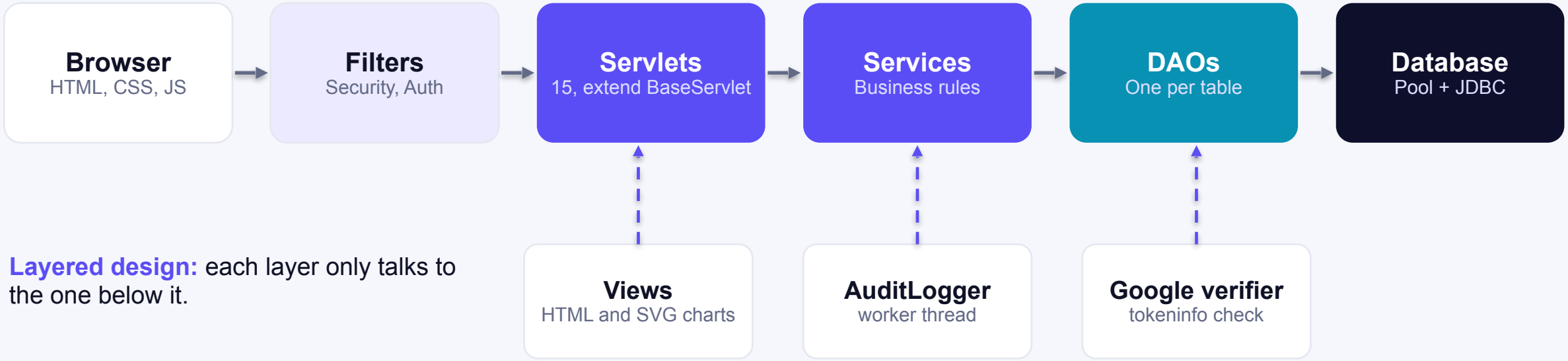
8

Install on a phone

Installable web app (PWA), CSV export, light and dark mode, 17-week calendar.

SOLUTION DESIGN

Architecture



Layered design: each layer only talks to the one below it.

- **Composition root:** AppContext wires every DAO and service
- **Thin servlets:** validate, call a service, redirect
- **Cross-cutting:** SecurityFilter (CSP, CSRF), AuthFilter
- **Observer:** StatsService cache cleared on every data change

SETUP

Technology and project structure

Java 17

records, switch expressions

Jakarta Servlet 5

servlets, filters, listeners

JDBC + H2

embedded, swappable to MySQL

Maven + Jetty

mvn jetty:run

JUnit 4

47 tests

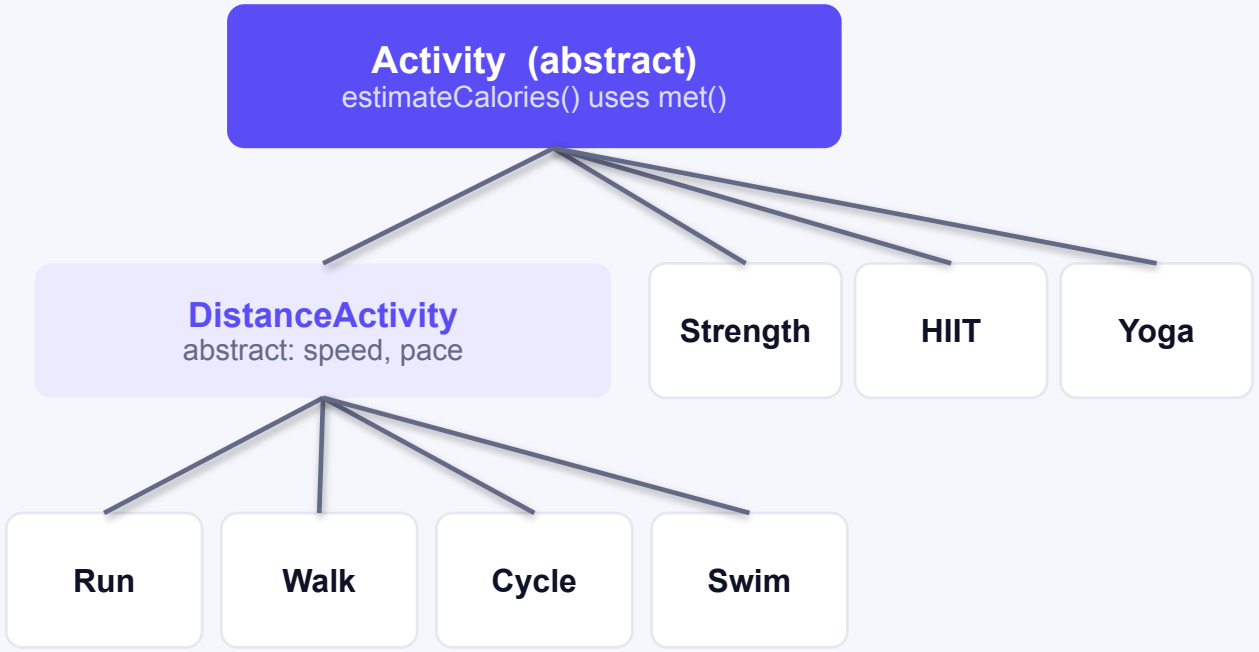
Google Identity

Sign in with Google

```
kinetic/
  pom.xml, README.md
  src/main/java/com/kinetic/
    model/       Activity hierarchy, enums
    exception/   KineticException tree
    db/          ConnectionPool, Database
    dao/         Dao<T,ID>, JdbcDao, 6 DAOs
    service/     Auth, Activity, Goal, Stats
    concurrent/ AuditLogger, OnlineUsers
    util/        PasswordHasher, LruCache...
    web/         filters, servlet/, view/
  src/main/resources/
    schema.sql, application.properties
  src/main/webapp/  css/ js/ WEB-INF/
  src/test/java/    47 JUnit tests
```

Run it: mvn jetty:run then open localhost:8080

OOP: inheritance, polymorphism, interfaces



Polymorphism: callers hold an Activity and never test its type.

```
a.applyCalories(user.getWeightKg());
// RunActivity.met()     -> speed based
// YogaActivity.met()    -> 1.8 + 0.5*effort
```

Interfaces and abstractions

- **Entity** common id contract
- **Trackable** goals work on any activity
- **Dao<T,ID>** generic CRUD contract
- **RowMapper<T>** ResultSet to object
- **GoogleTokenVerifier** real and fake versions
- **DataChangeListener** observer

Exception hierarchy

```
KineticException          (checked)
  ValidationException  AuthException
  NotFoundException
DataAccessException       (unchecked)
  wraps SQLException
```

Collections and generics

Dao<T extends Entity, ID>	Generic contract implemented by every DAO
LruCache<K, V>	Own generic cache: access-ordered LinkedHashMap + ReentrantLock
Collection<? extends Trackable>	Wildcard lets GoalMetric score any list of activities
EnumMap, TreeMap, TreeSet	Minutes by type, minutes by day, streak calculation
ConcurrentHashMap, key sets	Login throttle and online-users set
Optional, records, streams	findById results, Totals / Dashboard, groupingBy + toMap
Comparable<Activity>	Natural ordering by date, used by Collections.sort

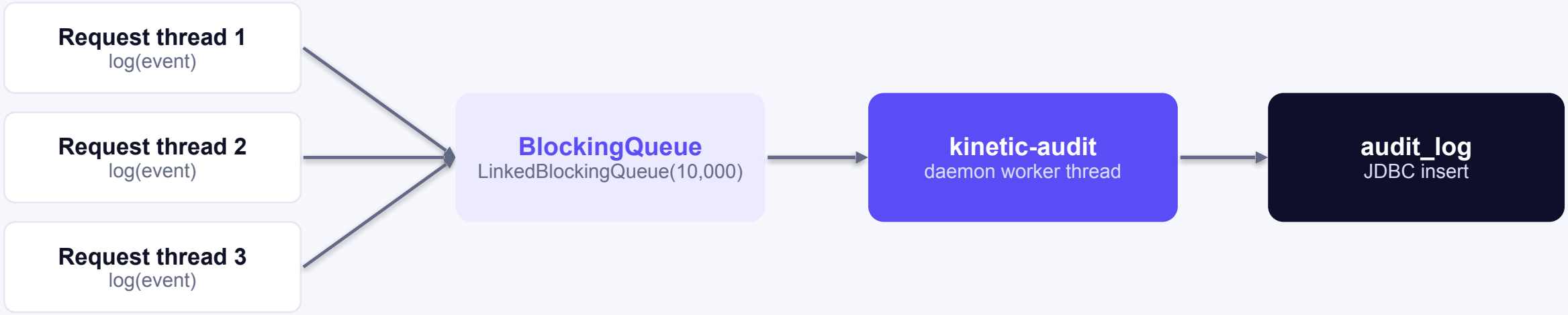
```
Map<LocalDate, List<Activity>> byWeek =
  window.stream().collect(
    groupingBy(a -> weekStart(a.date())));

Map<LocalDate, Integer> minutesByDay =
  window.stream().collect(toMap(
    Activity::date,
    Activity::durationMinutes,
    Integer::sum, TreeMap::new));

enum GoalMetric { WORKOUTS {
  double measure(Collection<
      ? extends Trackable> items)
  { return items.size(); } }, ...
```

Streaks use a TreeSet of active days and walk backwards from today.

Multithreading and synchronization



Where synchronization is used

- **AuditLogger:** producer/consumer, pages never wait for a DB write
- **ConnectionPool:** BlockingQueue of connections, 5 s wait
- **LruCache:** ReentrantLock around an access-ordered map
- **Transactions:** ThreadLocal connection per thread
- **Login throttle:** ConcurrentHashMap.compute is atomic
- **Online users:** ConcurrentHashMap key set
- **Shutdown:** poison pill flushes the queue
- **Proof:** ConcurrencyTest, 8 threads x 50 events = 400 rows

JDBC

Database design

users

id PK
email UNIQUE
display_name, password_hash
provider LOCAL | GOOGLE
google_sub UNIQUE
weight_kg, height_cm

1

N

activities

id PK
user_id FK
type, performed_on
duration_min, distance_km
intensity, calories
notes

goals

id PK
user_id FK
metric, target
UNIQUE(user_id, metric)

daily_logs

id PK, user_id FK
log_date UNIQUE per user
steps, water_ml, sleep_min

body_metrics

id PK, user_id FK
measured_on UNIQUE per user
weight_kg

audit_log

id PK, user_id
event, detail
created_at

activities, goals, daily_logs and body_metrics reference users with ON DELETE CASCADE.

DAOs: UserDao, ActivityDao, GoalDao, DailyLogDao, BodyMetricDao, AuditDao extend JdbcDao and implement Dao<T, ID>.

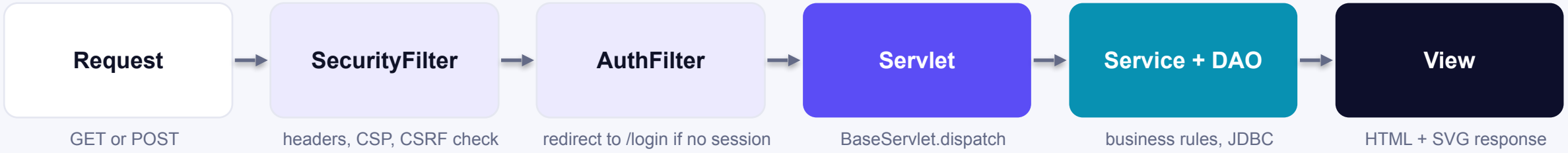
JDBC: pool, prepared statements, transactions

```
// JdbcDao: every query is parameterised
protected <T> List<T> query(String sql,
        RowMapper<T> mapper, Object... params) {
  return db.withConnection(c -> {
    try (PreparedStatement ps =
           c.prepareStatement(sql)) {
      bind(ps, params);
      try (ResultSet rs = ps.executeQuery()) {...}
```

```
// GoalService: all goals save, or none do
db.inTransaction(() -> {
  parsed.forEach((metric, target) -> {
    if (target <= 0) goals.deleteByMetric(..);
    else goals.save(new Goal(...));
  });
  return null;
});   // commit, or rollback on any exception
```

- **ConnectionPool:** fixed size, dynamic-proxy connections: close() returns to the pool
- **PreparedStatement only:** no string-built SQL, so no SQL injection
- **Transactions:** commit and rollback, nested calls join the open one
- **Batch insert:** addBatch / executeBatch seeds 16 weeks of demo data
- **Generated keys:** RETURN_GENERATED_KEYS sets the new id
- **Portable:** change db.url to use MySQL or PostgreSQL
- **Tested:** rollback test and cascade-delete test on H2

Servlets and web integration



BaseServlet: one error policy

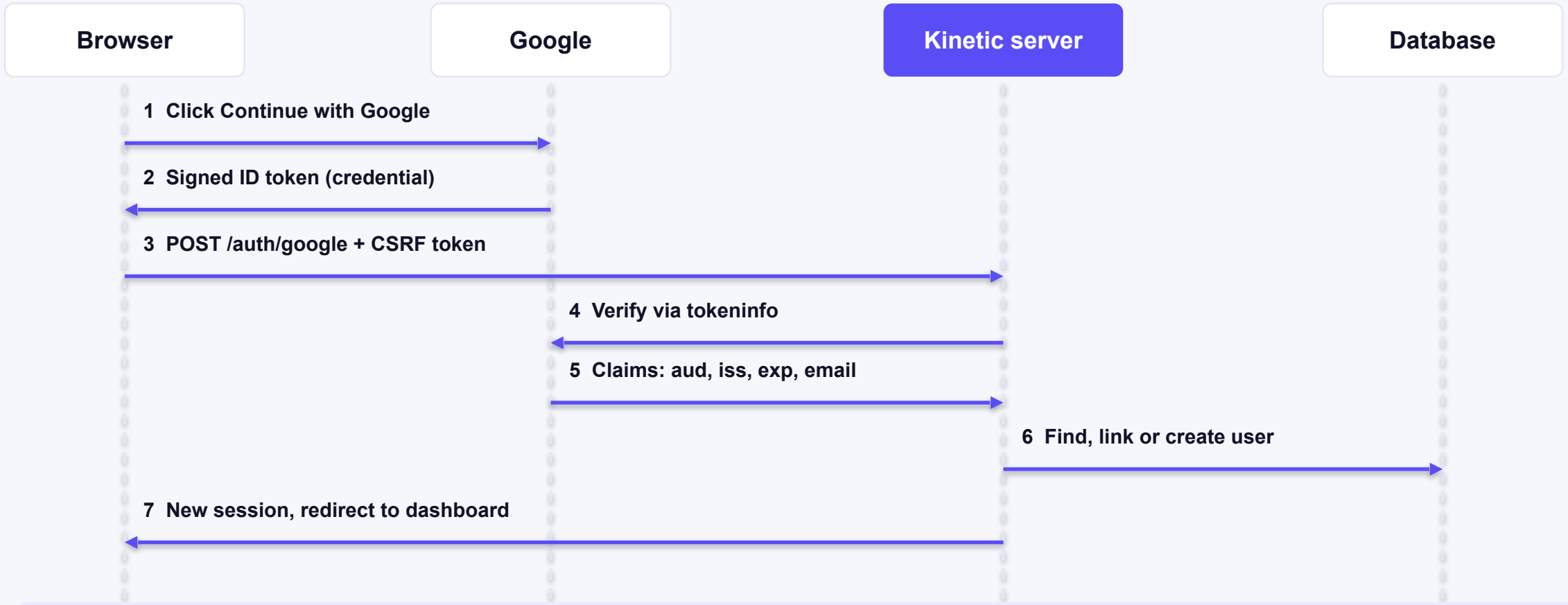
- ValidationException: message shown, form re-rendered
- AuthException: 401 with the email kept
- Anything else: logged, friendly 500 page
- Post, redirect, get with one-time flash messages

Servlet features used

- @WebServlet, @WebFilter, @WebListener
- HttpSession, session id rotation, timeout
- ServletContextListener starts the app and DB
- Error pages in web.xml, CSV file download
- /health JSON endpoint

AUTHENTICATION

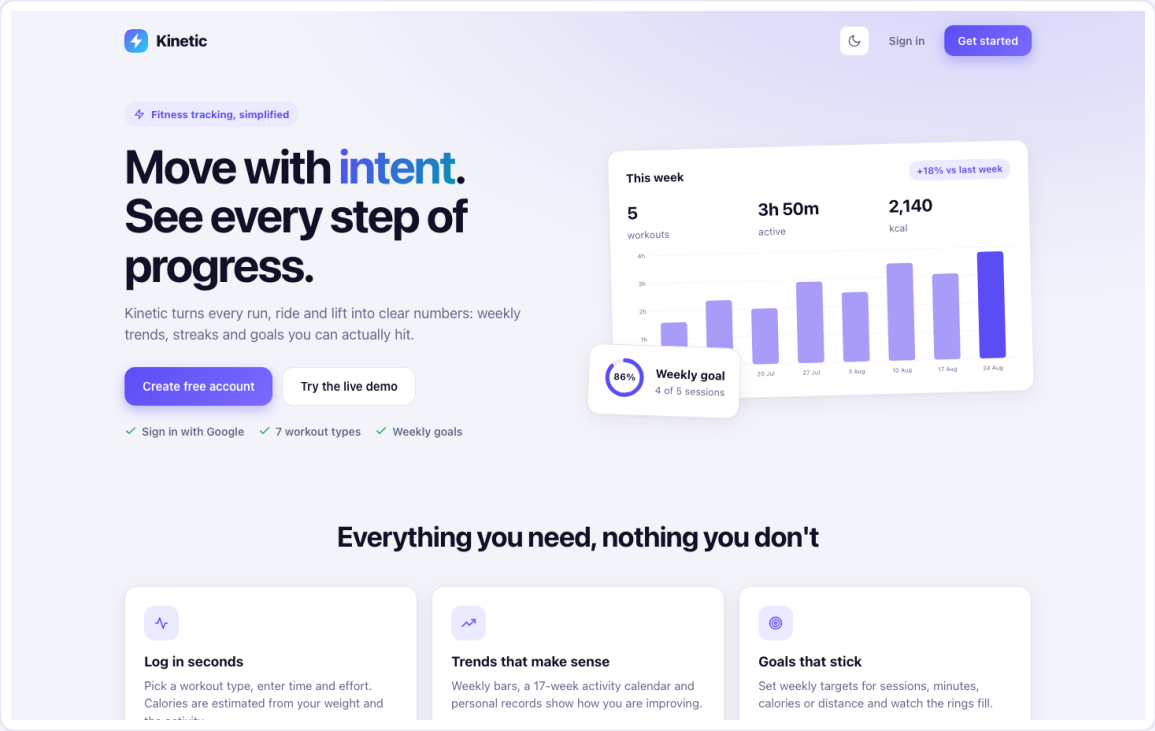
Sign in with Google



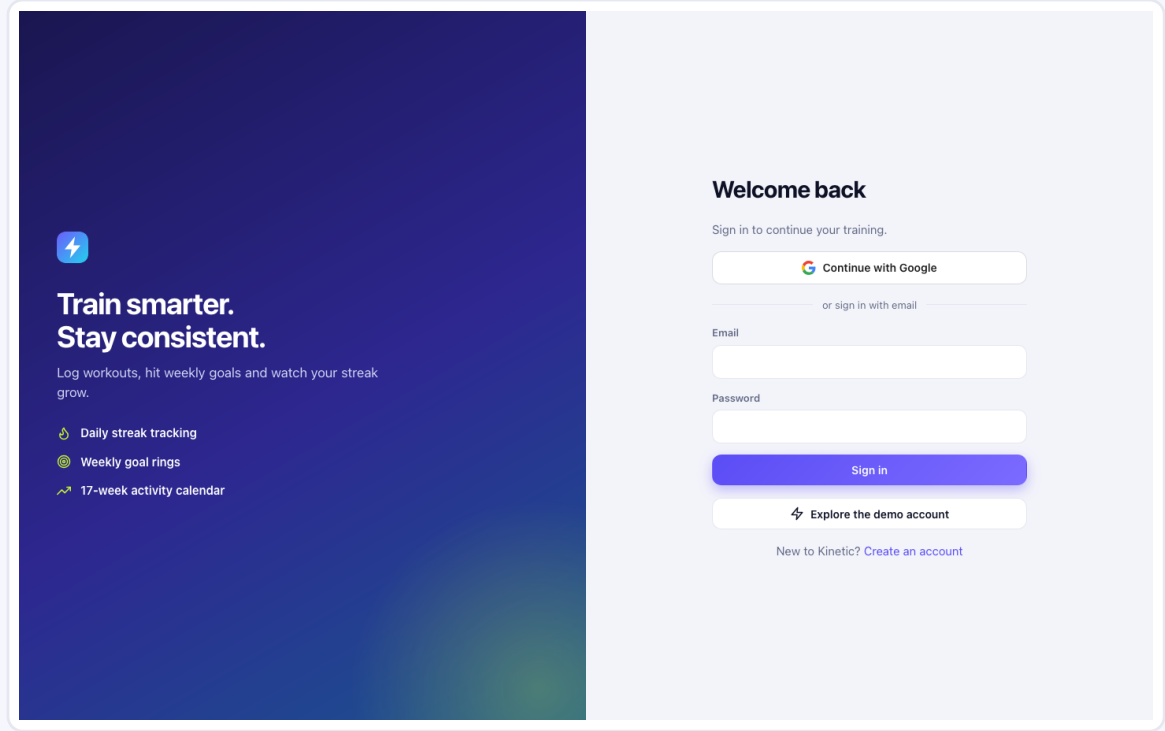
Server checks: audience equals our client ID, issuer is Google, token not expired, email verified. The browser is never trusted.

SCREENSHOTS

The application



Landing page

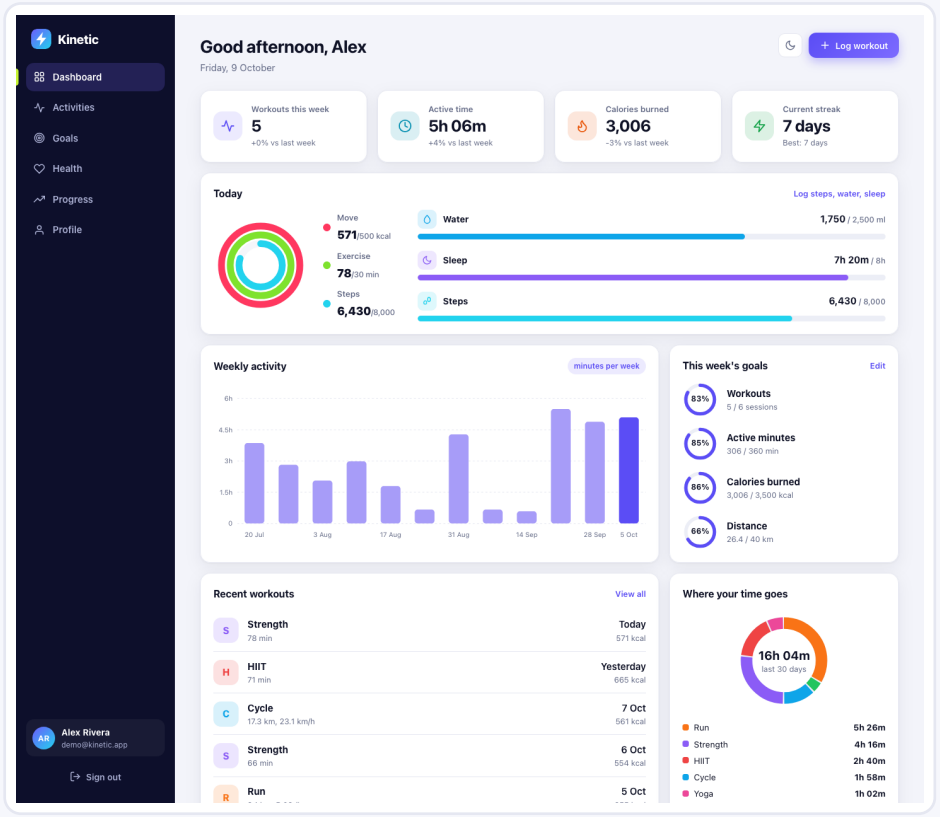


Sign in with Google or email

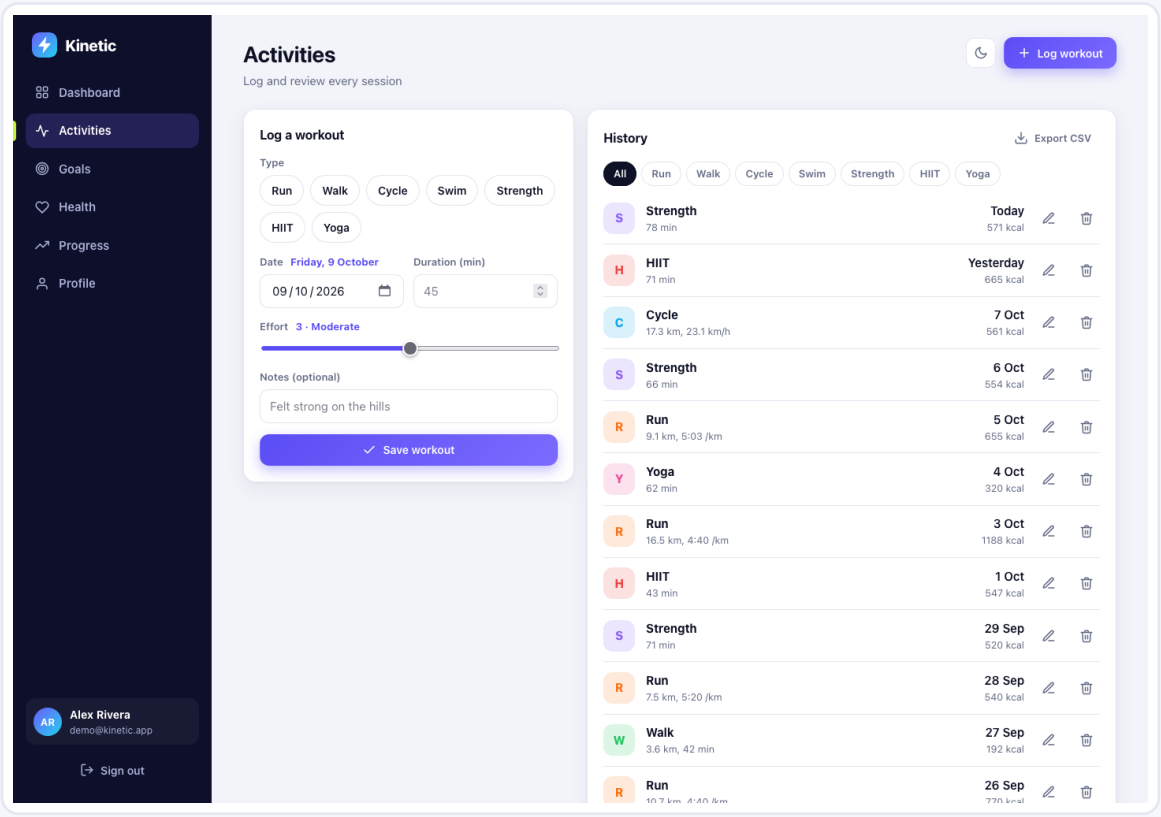
Public pages share the same design system: indigo brand colour, rounded cards, system fonts, light and dark themes.

SCREENSHOTS

Dashboard and activity log



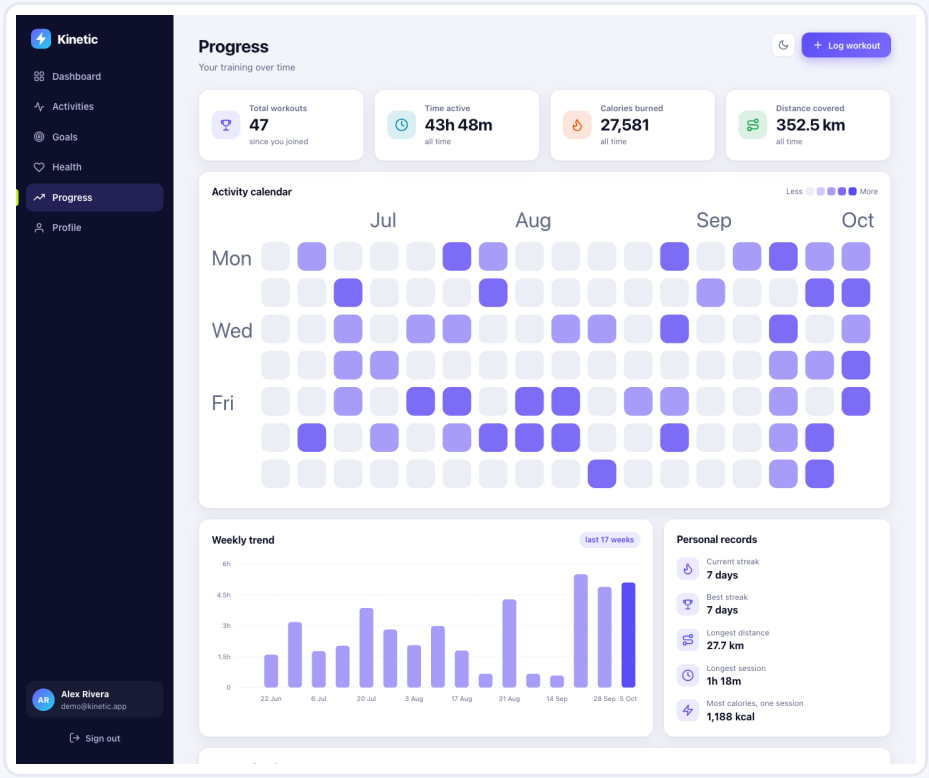
Dashboard: totals, 12-week chart, goal rings, breakdown



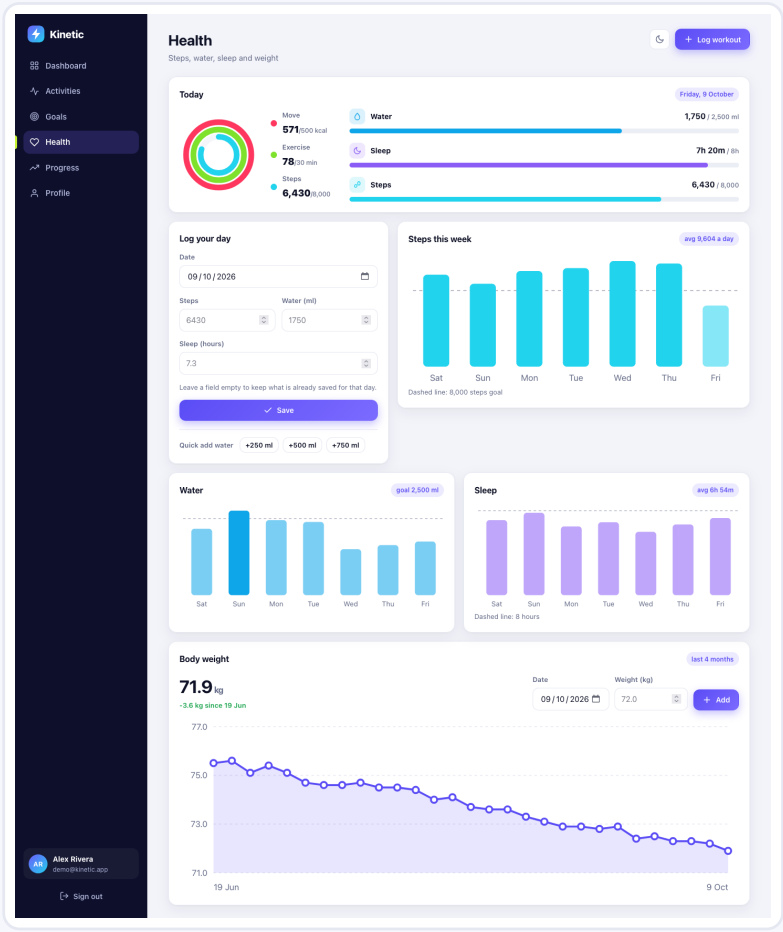
Activities: log form, filters, history, CSV export

SCREENSHOTS

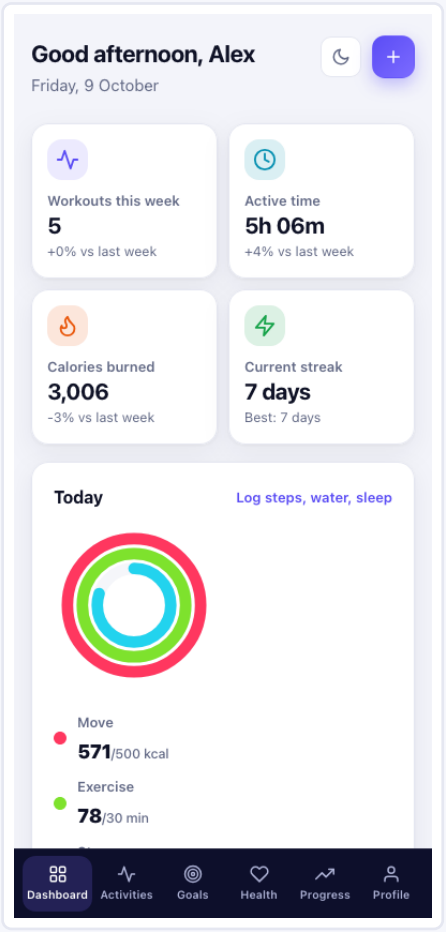
Health, progress and mobile



Progress: calendar heatmap, records



Health: rings, steps, water, sleep, weight



Phone layout

QUALITY

Security and testing

Security

- PBKDF2 hashing with per-user salt
- PreparedStatement everywhere (no SQL injection)
- HTML escaping on all output (no XSS)
- CSRF token on every POST
- Content-Security-Policy and security headers
- 5 failed logins per 10 minutes locks the email
- Users can only touch their own data
- Server-side Google token verification

Testing: 47 JUnit tests, all passing

- Polymorphic calorie maths and summaries
- Streak logic, goal scoring
- Registration, login, lockout
- Google: create, link, reject forged token
- Google claim checks (audience, issuer, expiry)
- Workout, goal, health and edit flows on in-memory JDBC
- Transaction rollback, cascade delete
- Thread safety: pool, audit logger, cache

COVERAGE

How the project meets the marking rubric

Rubric area	Evidence in Kinetic
Problem understanding and design	Layered architecture, ER design, clear user problem and solution
Core Java concepts	Inheritance, polymorphism, interfaces, exceptions, collections, generics, threads
Database integration (JDBC)	Own pool, PreparedStatement, transactions, batch, 6 DAO classes
Servlets and web integration	15 servlets, 2 filters, listener, sessions, CSRF, Google login, PWA
Code quality	Modular packages, comments, central error policy, 47 tests

Thank you

Kinetic: move with intent.

Run mvn jetty:run then localhost:8080

Demo Try the live demo button, or demo@kinetic.app

